

Class Activity #7

October 31, 2024

Section: 1 Student ID: 22200622 Name: Im Yejin

- ✓ Please upload your answer sheet in LMS. The uploading file must be PDF.
- ✓ Late submission is not allowed.
- ✓ Deadline: 11pm, 11/5 (Tue)

1. Practice #3.

Implement the TCP-based file upload program in Practice #3 on page 46. Submit the source code along with screenshots of the execution results. (Skeleton codes: file_up_server.c and file_up_client.c)

The screenshot displays two source code files and their execution results in a terminal window.

file_up_client.c (Left Pane):

```

12 {
13     serv_addr.sin_family = AF_INET;
14     serv_addr.sin_addr.s_addr = inet_addr(argv[1]);
15     serv_addr.sin_port = htons(atoi(argv[2]));
16
17     connect(sd, (struct sockaddr*)&serv_addr, siz
18
19     // TODO: Send file name to server
20     write(sd, file_name, sizeof(file_name));
21
22     // TODO: Send file data (Hint: file_server.c
23     while (1){
24         read_cnt = fread((void*)buf, 1, sizeof(b
25         read_size += read_cnt;
26         if(read_cnt < 1024){
27             write(sd, buf, read_cnt);
28             break;
29         }
30         printf("Send %d bytes \n", read_size);
31         write(sd, buf, read_cnt);
32     }
33
34     printf("Send total %d bytes \n", read_size);
35
36     // TODO: shutdown
37 }
  
```

file_up_server.c (Right Pane):

```

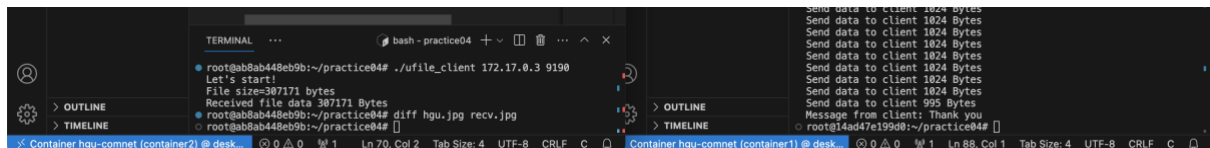
12 {
13     // TODO: Open file with received file name
14     fp = fopen(file_name, "wb");
15
16     // TODO: Receive file data from client
17     while((read_cnt=read(clnt_sd, buf, sizeof(buf))) != 0){
18         fwrite((void*)buf, 1, read_cnt, fp);
19     }
20
21     printf("Received file data: %s \n", file_name);
22
23     // TODO: Send complete message to client
24     write(clnt_sd, "Thank you", 10);
25
26     fclose(fp);
27     close(clnt_sd);
28     close(serv_sd);
29     return 0;
30 }
31
32 void error_handling(char *message)
33 {
34     fputs(message, stderr);
35     fputc('\n', stderr);
36     exit(1);
37 }
  
```

Terminal Output (Bottom):

```

root@14ad47e199d0:~# cd practice03
root@14ad47e199d0:~/practice03# gcc file_up_server.c -o file_up_server
root@14ad47e199d0:~/practice03# ./file_up_server 9190
Received file data: hgu2.jpg
root@14ad47e199d0:~/practice03# diff hgu.jpg hgu2.jpg
root@14ad47e199d0:~/practice03#
  
```

Implement the UDP-based file download program in Practice #4 on page 55. Submit the source code along with screenshots of the execution results. (Skeleton codes: ufile_server.c and ufile_client.c)



3. Practice #5.

Add SO_REUSEADDR option for TCP-based echo server in Practice #5 on page 66. (Skeleton codes: reuseadr_eserver.c)

The image shows two side-by-side screenshots of a code editor window, likely Visual Studio Code, running in a container environment. The left window displays the code for a client program, `echo_client.c`, and the right window displays the code for a server program, `reuseadr_eserver.c`.

Left Window (echo_client.c):

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/socket.h>
7
8 #define BUF_SIZE 1024
9 void error_handling(char *message);
10
11 int main(int argc, char *argv[])
12 {
13     int sock;
14     char message[BUF_SIZE];
15     int str_len, rcv_len, rcv_cnt;
16     struct sockaddr_in serv_addr;
17
18     if (argc != 3) {
19         printf("Usage : %s <IP> <port>\n", argv[0]);
20         exit(1);
21     }
22
23     sock = socket(PF_INET, SOCK_STREAM, 0);
24     if (sock == -1)
25         error_handling("socket() error");
26
27     memset(&serv_addr, 0, sizeof(serv_addr));
28     serv_addr.sin_family = AF_INET;
29     serv_addr.sin_addr.s_addr = inet_addr(argv[1]);
30     serv_addr.sin_port = htons(atoi(argv[2]));
31
32     // ... (rest of client code)

```

Right Window (reuseadr_eserver.c):

```

11
12 int main(int argc, char *argv[])
13 {
14     int serv_sock, clnt_sock;
15     char message[30];
16     int option, str_len;
17     struct sockaddr_in serv_addr, clnt_addr;
18
19     if (argc != 2) {
20         printf("Usage : %s <port>\n", argv[0]);
21         exit(1);
22     }
23
24     serv_sock = socket(PF_INET, SOCK_STREAM, 0);
25     if (serv_sock == -1)
26         error_handling("socket() error");
27
28     // TODO: Set socket option for REUSEADDR
29     option = sizeof(option);
30     option = TRUE;
31     setsockopt(serv_sock, SOL_SOCKET, SO_REUSEADDR, &option, option);
32
33     memset(&serv_addr, 0, sizeof(serv_addr));
34     serv_addr.sin_family = AF_INET;
35     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
36     serv_addr.sin_port = htons(atoi(argv[1]));
37
38     if (bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr))
39         error_handling("bind() error ");
40
41     // ... (rest of server code)

```

Both windows have a terminal at the bottom showing the execution of the programs. The left terminal shows the client connecting to the server and sending/receiving messages. The right terminal shows the server running and receiving connections from the client.